UNIVERSIDAD TECNOLÓGICA DEL PERÚ / INGENIERÍA

# GUÍA DE LABORATORIO 12

## Rutas en Angular y comunicación entre componentes

Angular Router + RouterLink + RouterOutlet + parámetros de ruta + input() / output() + navegación programática

| SEMANA 12<br>UNIDAD 3 | Meta de la sesión<br>Construir una SPA con navegación por rutas y componentes que intercambian datos de forma explícita y tipada. |
|---|---|

**Ruta de aprendizaje**

Semana 11: componentes y SCSS → Semana 12: navegación y comunicación → Semana 13: formularios y validación → Semana 14: consumo de servicios REST.

# 1. Propósito y logro de aprendizaje

En la semana anterior construimos componentes independientes y aprendimos a organizar la interfaz. Ahora necesitamos conectarlos: una aplicación real debe permitir que el usuario navegue entre vistas y que los componentes intercambien información sin quedar fuertemente acoplados.

## 1.1 Al finalizar el laboratorio podrás

- Explicar por qué una Single Page Application (SPA) necesita un router del lado del cliente.
- Definir rutas con app.routes.ts y registrar Angular Router con provideRouter(routes).
- Construir navegación sin recargar la página mediante RouterLink y RouterLinkActive.
- Usar RouterOutlet como punto de renderizado de las vistas activas.
- Crear rutas parametrizadas como /productos/:id y recuperar el parámetro desde ActivatedRoute.
- Enviar datos del componente padre al hijo con input() y notificar eventos del hijo al padre con output().
- Realizar navegación programática con Router.navigate().
- Implementar una ruta comodín ** para controlar URLs inexistentes.

## 1.2 Alcance de esta semana

**Importante**

No trabajaremos formularios complejos ni consumo HTTP todavía. Esos temas corresponden a las semanas 13 y 14. En esta práctica usaremos datos locales simulados para concentrarnos en routing y comunicación entre componentes.

# 2. Fundamentos: ¿qué problema resuelve Angular Router?

## 2.1 De una página tradicional a una SPA

En una aplicación web tradicional, cada cambio de URL suele solicitar al servidor un nuevo documento HTML. En una SPA, el navegador carga la aplicación una vez y el router decide qué componente mostrar según la URL, sin reemplazar toda la página.

| Concepto | Pregunta que responde | Ejemplo |
|---|---|---|
| Route / ruta | ¿Qué componente corresponde a esta URL? | /productos → Productos |
| RouterOutlet | ¿En qué lugar del layout se mostrará la vista activa? | <router-outlet /> |
| RouterLink | ¿Cómo navego sin forzar una recarga completa? | routerLink="/productos" |
| Parámetro de ruta | ¿Cómo identifico el recurso solicitado? | /productos/3 → id = 3 |
| Wildcard | ¿Qué hago si la URL no existe? | ** → NoEncontrado |

**Idea clave**

La URL se convierte en parte del estado de navegación. Esto permite compartir enlaces, usar atrás/adelante del navegador y llegar directamente a una vista concreta.

## 2.2 Las tres piezas esenciales

1. Las rutas describen el mapa URL → componente.
2. Los enlaces disparan navegación dentro de la SPA.
3. El outlet actúa como “ventana” donde Angular renderiza el componente correspondiente.

## 2.3 Comunicación entre componentes

Angular favorece un flujo de datos explícito. Para un componente padre y uno hijo usaremos dos canales complementarios:

| Dirección | API moderna | Qué transporta | Ejemplo |
|---|---|---|---|
| Padre → hijo | input() | Datos que el hijo necesita para renderizarse | [producto]="producto" |
| Hijo → padre | output() | Eventos que el padre debe atender | (verDetalle)="irDetalle($event)" |

**Regla práctica**

El hijo no debe "adivinar" qué hará el padre. El hijo comunica un evento; el padre decide la acción: navegar, actualizar un contador, abrir un modal, etc.

# 3. Preparación del entorno

## 3.1 Verifica las herramientas

**Terminal de VS Code**

```
node -v
npm -v
ng version
```

Para Angular 22 utiliza una versión de Node.js compatible. Una opción recomendable es Node.js 24.15 o superior. Si el comando ng no existe, instala Angular CLI.

```
npm install -g @angular/cli@latest
```

## 3.2 Crear el proyecto de la semana 12

```
ng new frontend-semana12 --routing --style=scss
cd frontend-semana12
code .
ng serve -o
```

**¿Qué hace --routing?**

Crea la estructura de routing desde el inicio y deja preparado app.routes.ts. La opción --style=scss mantiene la continuidad con el laboratorio de la semana 11.

## 3.3 Estructura que construiremos

```
src/app/
├── app.ts
├── app.html
├── app.scss
├── app.config.ts
├── app.routes.ts
├── models/
│   └── producto.ts
├── data/
│   └── productos.mock.ts
├── components/
│   └── producto-card/
└── pages/
    ├── inicio/
    ├── productos/
    ├── producto-detalle/
    └── no-encontrado/
```

## 3.4 Generar los componentes

```
ng g c pages/inicio
ng g c pages/productos
ng g c pages/producto-detalle
ng g c pages/no-encontrado
ng g c components/producto-card
```

**Convención Angular actual**

En Angular moderno el CLI genera componentes standalone y archivos con nombres como inicio.ts, inicio.html e inicio.scss. Por eso puede diferir de tutoriales antiguos que usan *.component.ts.

# 4. Crear el modelo y los datos locales

## 4.1 Modelo Producto

Crearemos una interfaz para que TypeScript verifique que todos los productos tengan la misma forma. Esto evita errores de nombres y tipos durante la comunicación entre componentes.

**src/app/models/producto.ts**

```
export interface Producto {
  id: number;
  nombre: string;
  categoria: string;
  precio: number;
  descripcion: string;
}
```

## 4.2 Fuente de datos simulada

**src/app/data/productos.mock.ts**

```
import { Producto } from '../models/producto';

export const PRODUCTOS: Producto[] = [
  {
    id: 1,
    nombre: 'Laptop Pro',
    categoria: 'Tecnología',
    precio: 3699,
    descripcion: 'Equipo para desarrollo y productividad.'
  },
  {
    id: 2,
    nombre: 'Monitor 27 pulgadas',
    categoria: 'Tecnología',
    precio: 1199,
    descripcion: 'Monitor QHD para trabajo profesional.'
  },
  {
    id: 3,
    nombre: 'Teclado mecánico',
    categoria: 'Accesorios',
    precio: 329,
    descripcion: 'Teclado compacto para programación.'
  }
];
```

**¿Por qué un archivo mock?**

Todavía no consumiremos el backend. Un mock permite practicar routing y comunicación sin mezclar conceptos de HttpClient, que corresponde a una semana posterior.

# 5. Configurar las rutas de la aplicación

## 5.1 Definir app.routes.ts

**src/app/app.routes.ts**

```
import { Routes } from '@angular/router';
import { Inicio } from './pages/inicio/inicio';
import { Productos } from './pages/productos/productos';
import { ProductoDetalle } from './pages/producto-detalle/producto-detalle';
import { NoEncontrado } from './pages/no-encontrado/no-encontrado';

export const routes: Routes = [
  { path: '', component: Inicio },
  { path: 'productos', component: Productos },
  { path: 'productos/:id', component: ProductoDetalle },
  { path: '**', component: NoEncontrado }
];
```

### 5.2 Verificar provideRouter(routes)

El proyecto creado con --routing normalmente ya contiene esta configuración. No reemplaces otros providers que Angular haya generado; solo verifica que provideRouter(routes) exista.

**Concepto mínimo de app.config.ts**

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)]
};
```

## 6. Construir el layout y la navegación

### 6.1 Componente raíz

**src/app/app.ts**

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterLink, RouterLinkActive, RouterOutlet],
  templateUrl: './app.html',
  styleUrl: './app.scss'
})
export class App {}
```

### 6.2 Plantilla principal

**src/app/app.html**

```
<header class="topbar">
  <div>
    <strong>DevStore</strong>
    <span>Laboratorio Angular</span>
  </div>

  <nav>
    <a routerLink="/" routerLinkActive="active"
       [routerLinkActiveOptions]="{ exact: true }">Inicio</a>
    <a routerLink="/productos" routerLinkActive="active">Productos</a>
  </nav>
</header>

<main class="container">
  <router-outlet />
</main>
```

**No uses href para rutas internas**

routerLink permite que Angular cambie la vista sin recargar toda la aplicación. Un enlace tradicional con href puede provocar una navegación completa del navegador.

### 6.3 Estilos del layout

**src/app/app.scss**

```
$primary: #0f3d66;
$accent: #168aad;
$surface: #ffffff;
$background: #f4f7fb;

:host {
  display: block;
  min-height: 100vh;
  background: $background;
}

.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: $primary;
  color: white;

  nav { display: flex; gap: 1rem; }
  a { color: #d9e8f5; text-decoration: none; }
  a.active { color: white; font-weight: 700; border-bottom: 2px solid $accent; }
}

.container {
  width: min(1100px, 92%);
  margin: 2rem auto;
}
```

## 7. Crear las páginas Inicio y No Encontrado

### 7.1 Inicio

**src/app/pages/inicio/inicio.ts**

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-inicio',
  imports: [RouterLink],
  templateUrl: './inicio.html',
  styleUrl: './inicio.scss'
})
export class Inicio {}
```

**src/app/pages/inicio/inicio.html**

```
<section class="hero">
  <p class="eyebrow">DESARROLLO WEB INTEGRADO</p>
  <h1>Navegación y comunicación en Angular</h1>
  <p>Explora el catálogo y observa cómo colaboran los componentes.</p>
  <a class="btn" routerLink="/productos">Ver productos</a>
</section>
```

**src/app/pages/inicio/inicio.scss**

```
.hero {
  padding: 3rem;
  border-radius: 1.25rem;
  background: linear-gradient(135deg, #0f3d66, #168aad);
  color: white;

  .eyebrow { opacity: .8; font-weight: 700; }
  h1 { margin: .4rem 0 1rem; font-size: clamp(2rem, 4vw, 3.2rem); }
}

.btn {
  display: inline-block;
  margin-top: 1rem;
  padding: .75rem 1rem;
  border-radius: .6rem;
  background: white;
  color: #0f3d66;
  text-decoration: none;
  font-weight: 700;
}
```

### 7.2 Página 404

**src/app/pages/no-encontrado/no-encontrado.html**

```
<section>
  <p class="eyebrow">ERROR 404</p>
  <h1>Ruta no encontrada</h1>
  <p>La dirección solicitada no existe dentro de esta aplicación.</p>
</section>
```

**Prueba inmediata**

Con ng serve ejecutándose, abre http://localhost:4200/una-ruta-que-no-existe. Debe aparecer NoEncontrado gracias a la ruta **.

## 8. Comunicación padre → hijo con input()

### 8.1 ProductoCard recibirá un Producto

**src/app/components/producto-card/producto-card.ts**

```
import { Component, input, output } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { Producto } from '../../models/producto';

@Component({
  selector: 'app-producto-card',
  imports: [CurrencyPipe],
  templateUrl: './producto-card.html',
  styleUrl: './producto-card.scss'
})
export class ProductoCard {
  readonly producto = input.required<Producto>();
  readonly verDetalle = output<number>();
  readonly favorito = output<number>();

  solicitarDetalle(): void {
    this.verDetalle.emit(this.producto().id);
  }

  marcarFavorito(): void {
    this.favorito.emit(this.producto().id);
  }
}
```

input.required<Producto>() declara que el componente hijo necesita obligatoriamente un Producto. Angular puede verificar este contrato en tiempo de compilación. Como input() devuelve una señal, su valor se lee con producto().

### 8.2 Plantilla del hijo

**src/app/components/producto-card/producto-card.html**

```
<article class="card">
  <span class="badge">{{ producto().categoria }}</span>
  <h2>{{ producto().nombre }}</h2>
  <p>{{ producto().descripcion }}</p>
  <strong>{{ producto().precio | currency:'PEN':'symbol':'1.2-2' }}</strong>

  <div class="actions">
    <button type="button" (click)="solicitarDetalle()">Ver detalle</button>
    <button type="button" class="ghost" (click)="marcarFavorito()">Favorito</button>
  </div>
</article>
```

**src/app/components/producto-card/producto-card.scss**

```
.card {
  background: white;
  border: 1px solid #d9e2ec;
  border-radius: 1rem;
  padding: 1.25rem;
  box-shadow: 0 10px 25px rgba(15, 61, 102, .08);

  .badge { color: #168aad; font-size: .8rem; font-weight: 700; }
  h2 { margin: .5rem 0; color: #102a43; }
  .actions { display: flex; gap: .6rem; margin-top: 1rem; }
  button { cursor: pointer; padding: .6rem .85rem; border-radius: .5rem; border: 0; background: #0f3d66;
color: white; }
  button.ghost { background: #eaf4fb; color: #0f3d66; }
}
```

## 9. Comunicación hijo → padre con output()

### 9.1 Componente Productos: el padre

**src/app/pages/productos/productos.ts**

```
import { Component, inject, signal } from '@angular/core';
import { Router } from '@angular/router';
import { ProductoCard } from '../../components/producto-card/producto-card';
import { PRODUCTOS } from '../../data/productos.mock';

@Component({
  selector: 'app-productos',
  imports: [ProductoCard],
  templateUrl: './productos.html',
  styleUrl: './productos.scss'
})
export class Productos {
  private readonly router = inject(Router);

  readonly productos = signal(PRODUCTOS);
  readonly favoritos = signal(0);

  irDetalle(id: number): void {
    this.router.navigate(['/productos', id]);
  }

  registrarFavorito(id: number): void {
    console.log('Favorito recibido desde el hijo:', id);
    this.favoritos.update(valor => valor + 1);
  }
}
```

### 9.2 Plantilla del padre

**src/app/pages/productos/productos.html**

```
<section class="page-heading">
  <div>
    <p class="eyebrow">CATÁLOGO</p>
    <h1>Productos</h1>
  </div>
  <p>Favoritos seleccionados: <strong>{{ favoritos() }}</strong></p>
</section>

<div class="grid">
  @for (producto of productos(); track producto.id) {
    <app-producto-card
      [producto]="producto"
      (verDetalle)="irDetalle($event)"
      (favorito)="registrarFavorito($event)" />
  }
</div>
```

**src/app/pages/productos/productos.scss**

```
.page-heading {
  display: flex;
  justify-content: space-between;
  align-items: end;
  margin-bottom: 1.5rem;
}

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1rem;
}
```

**Lee el flujo**

El padre entrega [producto] al hijo. El hijo emite (verDetalle) o (favorito). El padre escucha el evento y ejecuta la acción. Esa separación hace que ProductoCard sea reutilizable.

## 10. Navegación programática y parámetros de ruta

Al pulsar “Ver detalle”, el hijo emite el id. El padre recibe ese id y ejecuta Router.navigate(["/productos", id]). Si id = 2, Angular navega a /productos/2.

### 10.1 Recuperar :id con ActivatedRoute

**src/app/pages/producto-detalle/producto-detalle.ts**

```
import { Component, inject } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { RouterLink, ActivatedRoute } from '@angular/router';
import { PRODUCTOS } from '../../data/productos.mock';

@Component({
  selector: 'app-producto-detalle',
  imports: [CurrencyPipe, RouterLink],
  templateUrl: './producto-detalle.html',
  styleUrl: './producto-detalle.scss'
})
export class ProductoDetalle {
  private readonly route = inject(ActivatedRoute);

  readonly id = Number(this.route.snapshot.paramMap.get('id'));
  readonly producto = PRODUCTOS.find(item => item.id === this.id);
}
```

**src/app/pages/producto-detalle/producto-detalle.html**

```
@if (producto; as p) {
  <section class="detail">
    <p class="eyebrow">{{ p.categoria }}</p>
    <h1>{{ p.nombre }}</h1>
    <p>{{ p.descripcion }}</p>
    <h2>{{ p.precio | currency:'PEN':'symbol':'1.2-2' }}</h2>
    <a routerLink="/productos">← Volver al catálogo</a>
  </section>
} @else {
  <section>
    <h1>Producto no encontrado</h1>
    <a routerLink="/productos">Volver al catálogo</a>
  </section>
}
```

**Detalle técnico**

snapshot es suficiente para este laboratorio porque la vista se abre con una URL nueva. Más adelante puedes reaccionar a cambios del parámetro mediante paramMap como Observable o convertirlo a Signal.

# 11. Ejecutar y comprobar la integración

## 11.1 Levantar la aplicación

```
ng serve
```

## 11.2 Matriz de pruebas manuales

| Prueba | Acción | Resultado esperado |
|---|---|---|
| Inicio | Abrir http://localhost:4200/ | Se visualiza la página Inicio dentro de router-outlet. |
| Navegación | Pulsar Productos | Cambia a /productos sin recargar toda la página. |
| Ruta activa | Observar menú | Productos aparece con clase active. |
| Input | Ver tarjetas | Cada ProductoCard muestra el objeto recibido del padre. |
| Output favorito | Pulsar Favorito dos veces | El padre incrementa el contador a 2. |
| Output + Router | Pulsar Ver detalle en id 2 | Navega a /productos/2. |
| Parámetro | Abrir /productos/3 | Muestra Teclado mecánico. |
| ID inexistente | Abrir /productos/999 | Muestra Producto no encontrado. |
| Wildcard | Abrir /xyz | Muestra la página 404. |
| Historial | Usar atrás/adelante | La navegación responde al historial del navegador. |

## 11.3 ¿Cómo comprobar que no hubo recarga completa?

Abre las herramientas de desarrollo del navegador, pestaña Network. Navega entre Inicio y Productos. La aplicación debería cambiar de vista sin volver a descargar index.html en cada clic interno con routerLink.

# 12. Errores frecuentes y cómo diagnosticarlos

| Síntoma | Causa probable | Corrección |
|---|---|---|
| La vista no aparece | Falta <router-outlet /> | Importa RouterOutlet y agrega el outlet al template raíz. |
| routerLink no funciona | No está importado RouterLink | Agrega RouterLink en imports del componente standalone. |
| URL /productos no coincide | Ruta mal escrita | Revisa app.routes.ts y el valor de routerLink. |
| El hijo muestra undefined | Input no enviado o no tipado | Usa input.required<Producto>() y [producto]="producto". |
| El padre no recibe el evento | Nombre del output distinto | Verifica output() y el evento entre paréntesis. |
| /productos/2 muestra error | Conversión de id incorrecta | Convierte el parámetro a number antes de buscar. |
| 404 no se activa | Wildcard está antes que otras rutas | La ruta ** debe ir al final del arreglo. |
| Página se recarga | Se utilizó href en lugar de routerLink | Usa RouterLink para navegación interna. |

# 13. Actividad de consolidación

Amplía la miniaplicación sin consultar todavía un backend. El propósito es demostrar que comprendes navegación, parámetros y comunicación.

### Reto A — nueva ruta Categorías

- Crea el componente pages/categorias.
- Agrega la ruta /categorias.
- Incluye un enlace en el menú principal.
- Muestra las categorías únicas de PRODUCTOS.

### Reto B — tarjeta reutilizable

- Agrega un input destacado = input(false).
- Cuando destacado sea true, aplica una clase visual especial.
- Desde Productos, marca como destacado al producto cuyo precio sea mayor a S/ 1,000.

### Reto C — evento adicional

- Agrega output<number>() llamado agregarComparacion.
- Emite el id desde ProductoCard.
- El padre debe almacenar y mostrar cuántos productos se añadieron a comparación.

### Reto D — consulta directa

Escribe manualmente en el navegador /productos/1 y /productos/999. Explica por qué ambas URLs llegan al mismo componente pero producen resultados distintos.

**Criterio de calidad**

No copies lógica del padre dentro del hijo. Mantén ProductoCard enfocado en presentar información y emitir eventos. El padre debe orquestar navegación y estado de la página.

# 14. Preguntas de reflexión

1. ¿Cuál es la diferencia entre cambiar de ruta con routerLink y usar un enlace href tradicional?
2. ¿Qué responsabilidad cumple RouterOutlet?
3. ¿Por qué la ruta ** debe colocarse al final?
4. ¿Qué ventaja ofrece input.required() frente a un input opcional?
5. ¿Por qué output() reduce el acoplamiento entre el componente hijo y el padre?
6. ¿Cuándo conviene usar un parámetro de ruta en lugar de un input?
7. ¿Qué parte del flujo de esta práctica será reemplazada por datos provenientes del backend en la semana 14?

# 15. Cierre conceptual

Hasta la Semana 11 teníamos componentes aislados. Después de este laboratorio ya contamos con una aplicación navegable: la URL selecciona una vista, los componentes padre entregan datos a sus hijos y los hijos notifican eventos al padre. Esta estructura es la base para integrar formularios, servicios HTTP y finalmente el back-end Spring Boot.

**Mapa mental final**

URL → Route → Component → RouterOutlet. Padre → input() → Hijo. Hijo → output() → Padre. Padre → Router.navigate() → nueva URL.

### 15.1 Preparación para la Semana 13

La siguiente sesión incorporará procesamiento de formularios, validación de datos y manejo de errores. Conserva este proyecto: la navegación creada hoy servirá para incorporar pantallas de registro y edición.

## 18. Referencias técnicas de apoyo


- Angular Documentation — Routing overview y Router reference (angular.dev/guide/routing).
- Angular Documentation — Component inputs: input() e input.required().
- Angular Documentation — Custom events with outputs: output().
- Angular Documentation — Versioning and releases / Version compatibility.